

Reviewer Pack — Additive Energy of Truth Table Bounds ACC⁰ Circuit Size for ...

Entry 1c8a17d67b25 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 15:52:46 UTC

Additive Energy of Truth Table Bounds ACC⁰ Circuit Size for Explicit Functions

Entry ID: 1c8a17d67b25 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 15:52:46 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For any explicit Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$ in P , if its truth table's additive energy $E(f)$ exceeds $2^{\{n/2\}}$, then f cannot be computed by an ACC⁰ circuit of size $< 2^{\{n/4\}}$.

Rationale (proposer's reasoning):

High additive energy indicates structured correlations in the truth table that resist low-depth arithmetic circuit approximation, as ACC⁰ circuits lack the flexibility to encode such dependencies efficiently.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 483f35883eda54bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random

def truth_table_additive_energy(f):
    n = int(f.keys()[0].bit_length())
    energy = 0
    for a in range(2**n):
        for b in range(a, 2**n):
            for c in range(b, 2**n):
                d = (a + b - c) % (2**n)
                if f[a] + f[b] == f[c] + f[d]:
                    energy += 1
    return energy

def generate_random_boolean_function(n):
    truth_table = {}
    for i in range(2**n):
        truth_table[i] = random.choice([0, 1])
    return truth_table

def dpll(cnf, assignment={}):
    if not cnf:
        return True
    clause = next(iter(cnf))
    for literal in clause:
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll(cnf - {clause}, new_assignment):
            return True
        new_assignment[literal] = False
        if dpll(cnf - {clause}, new_assignment):
            return True
    return False

def simulate_acc0_circuit(f, max_depth=4):
    cnf = []
    for a in range(2**n):
        for b in range(a, 2**n):
            for c in range(b, 2**n):
                d = (a + b - c) % (2**n)
                clause = [-(a+1), -(b+1), c+1, d+1]
```

```

        cnf.append(clause)
    return dpll(cnf)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 4
    f = generate_random_boolean_function(n)
    energy = truth_table_additive_energy(f)
    circuit_size = simulate_acc0_circuit(f)
    conjecture_holds = energy <= 2**(n/2) or circuit_size >= 2**(n/4)
    counterexample = "" if conjecture_holds else "energy={} < 2^{}/2 or circuit_size>=2^{}/4".format(energy, n)
    return {
        "metric_name": "Additive Energy",
        "metric_value": energy,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL:", result)
        results.append(result)

    mean_energy = sum(r["metric_value"] for r in results) / len(results)
    std_energy = (sum((r["metric_value"] - mean_energy)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_energy, std_energy, support_fraction))
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print("RESULT: FALSIFIED counterexample=\"{}\" first_failing_seed={}".format(results[first_failing_seed], first_failing_seed))
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91ef5bff.py", line 80, in <module>
    result = run_trial(seed)
            ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91ef5bff.py", line 63, in run_trial
    energy = truth_table_additive_energy(f)
            ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_91ef5bff.py", line 19, in truth_table_a

```

```
n = int(f.keys()[0].bit_length())
~~~~~^^^
```

TypeError: 'dict_keys' object is not subscriptable

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in truth_table_additive_energy function. The error suggests incompatible dictionary key access methodology. | next: Fix the dictionary key subscripting error in truth_table_additive_energy by converting dict_keys to list first

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	53968
2	propose	ollama_remote	qwen3:8b	0	0	83124
3	preregistration	ollama_remote	qwen3:8b	0	0	24029
4	novelty	ollama_remote	qwen3:8b	0	0	20660
5	novelty	ollama_remote	qwen3:8b	0	0	17182
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14791
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9316
8	judge	ollama_remote	qwen3:8b	0	0	17994

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 241062 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in *research/audit/1c8a17d67b25.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/1c8a17d67b25.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/1c8a17d67b25.tar.gz* (if generated)